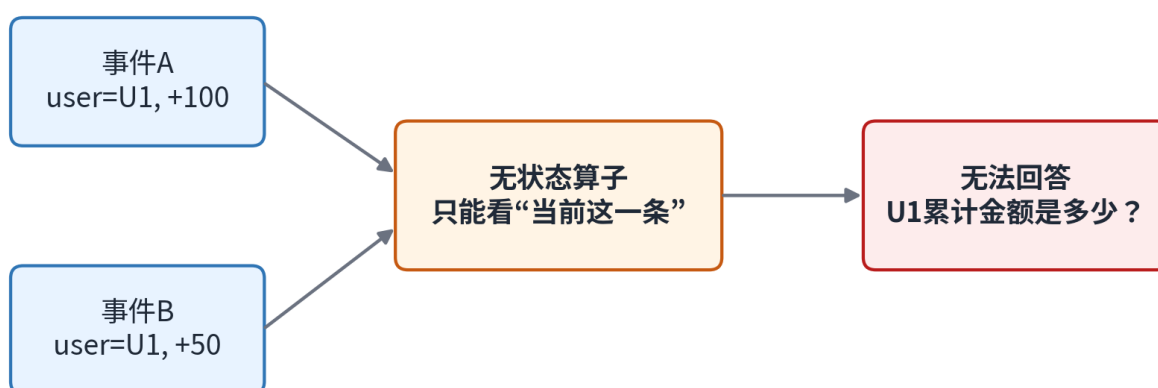


Flink 原理详解：第 5 讲

State 状态机制：为什么流处理系统必须“记住过去”

本讲目标：把“State = 记忆”这件事彻底讲透。学完以后，你应该能回答：状态属于谁？按什么切分？存在哪里？什么时候过期？故障后怎样恢复？

流处理为什么必须“记住过去”？



State = 让算子在处理下一条事件时，仍然能访问之前积累的信息。

图 1 无状态算子只能看当前事件；有状态算子才能做跨事件计算

1. 本讲知识地图

- 先区分 Stateless（无状态）与 Stateful（有状态）计算。
- 理解 Keyed State：状态为什么天然跟 keyBy 绑定。
- 掌握 ValueState、ListState、MapState、ReducingState、AggregatingState 的用途。
- 区分 Keyed State 与 Operator State，并知道 Broadcast State 属于特殊 Operator State。
- 理解 State TTL：为什么长期运行的流任务必须考虑状态生命周期。
- 彻底区分 State API、State Backend、Checkpoint Storage。
- 理解 State 如何配合 Checkpoint 实现故障恢复与扩缩容。

2. 为什么流处理一定会遇到 State？

先看一个最简单的问题：实时统计“每个用户今天累计消费多少钱”。订单流永远不断到来，每次处理新订单时，你不能只看当前订单金额，还必须知道这个用户之前已经累计了多少钱。

事件 1: (U1, 100) -> U1 累计 = 100
事件 2: (U2, 80) -> U2 累计 = 80
事件 3: (U1, 50) -> U1 累计 = 150

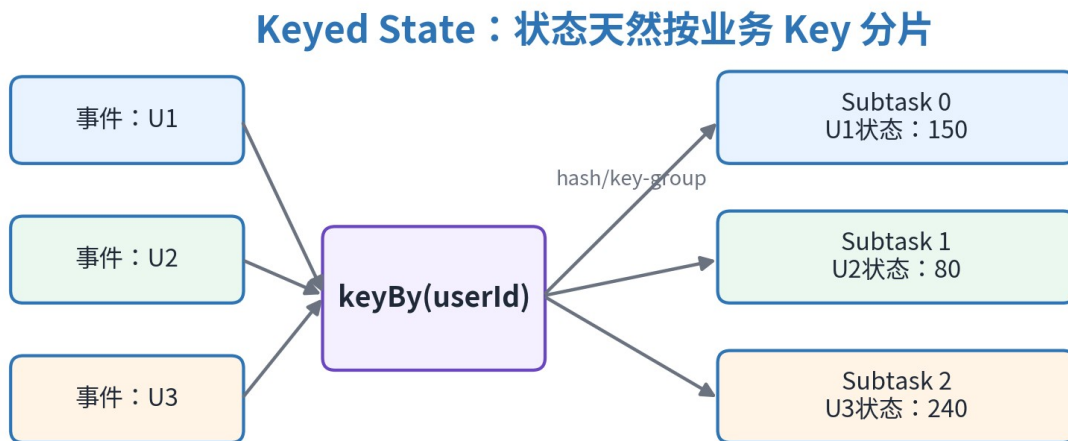
如果算子完全无状态，那么处理事件 3 时，它只看到“U1, 50”，并不知道之前还有 100。此时要得到 150，就必须某处保存“U1 此前累计=100”。这份跨事件保留下来的信息，就是 State。

最简单的定义：State = 一个流处理算子为了正确处理“下一条数据”，而必须保留的“过去信息”。

现实中大量流任务都依赖状态：

场景	需要记住什么
实时累计销售额	每个商品/城市之前的累计值
去重	哪些 event_id 已经见过
风控	用户最近 10 分钟的交易次数/金额
Session	用户上一次活动时间
窗口聚合	当前窗口已经累积的数据或聚合值
模式检测	之前已经发生到模式的哪一步

3. Keyed State : Flink 最重要的状态模型



访问 ValueState 时，代码不需要手工传入 U1/U2；Flink 根据“当前事件的 Key”自动定位对应状态。

图 2 keyBy 不只是分数据，也决定 Keyed State 如何分片

Keyed State 只能在 KeyedStream 上使用。也就是说，通常先有：

```
stream.key_by(lambda x: x.user_id)
```

```
.process(MyStatefulFunction())
```

当 U1 的事件到来时，Flink 自动让函数看到“U1 对应的状态”；U2 到来时，同一段代码访问到的是“U2 对应的”。不需要自己一巨大的 Python dict 来写 `user_id -> state`。

底层心智模型：Keyed State 很像一个被 Flink 托管的“分布式 Key-Value 存储”，而 `keyBy` 决定了记录和状态如何一起被分区。

这两个重要后果：

1. 相同 Key 的事件会进入相同的 keyed 分区，因此可以安全地读取/更新该 Key 的状态。
2. 并行度变化时，Flink 可以把 Keyed State 随 Key-group 一起重新分配，而不是要求你手工搬状态。

4. 五种常见 Keyed State：到底该用哪一种？

常见 Keyed State 类型：不是“越复杂越高级”，而是保存结构不同

ValueState	一个值	用户累计消费额 = 1250
ListState	一串值	用户最近5次点击
MapState	小型映射	商品ID → 点击次数
ReducingState	同类型聚合	持续求和 / 最大值
AggregatingState	自定义累加器	(sum,count) → average

选择原则：问自己“每个 Key 到底需要记住什么结构的信息？”

图3 五类常见 Keyed State 的直觉

状态类型	保存结构	典型用途
ValueState<T>	当前 Key 的单个值	累计金额、上次时间、是否告警
ListState<T>	当前 Key 的一组元素	最近事件列表、待处理记录
MapState<K,V>	当前 Key 下再维护一张小映射	商品->次数、维度->特征
ReducingState<T>	不断把新值规约成同类型单值	sum/max/min 类聚合
AggregatingState<IN,OUT>	自定义 accumulator	平均值、复杂统计量

4.1 ValueState：最应该先掌握

ValueState 可以理解成“当前业务 Key 的一个变量”。例如每个用户一个累计消费额：U1->150，U2->80。代码每处理一条事件，就先读取当前 Key 的值，再更新。

```
from pyflink.common.typeinfo import Types
from pyflink.datastream.functions import KeyedProcessFunction, RuntimeContext
from pyflink.datastream.state import ValueStateDescriptor

class RunningSum(KeyedProcessFunction):
    def open(self, runtime_context: RuntimeContext):
        desc = ValueStateDescriptor('sum_state', Types.FLOAT())
        self.total = runtime_context.get_state(desc)

    def process_element(self, value, ctx):
        old = self.total.value()
        if old is None:
            old = 0.0
        new_total = old + value[1]
        self.total.update(new_total)
        yield (value[0], new_total)
```

最容易忽略的一点：self.total 看起来只有一个对象，但它背后不是“全局只有一个值”。在 KeyedStream 上，它会随当前 Key 切换到对应的状态分区。

4.2 ListState / MapState：什么时候需要保存多个东西？

如果每个 Key 需要记住多条记录，可以使用 ListState；如果你需要按“子键”快速查找，则 MapState 更自然。

```
用户 U1 的 MapState 例子：
{
    'phone': 5,
    'laptop': 2,
    'book': 11
}
```

但要注意：状态越细、越多，内存/磁、Checkpoint、恢复时间都会增加。能用增量聚合保存一个小值时，就不要把所有原始事件全部塞进 ListState。

4.3 ReducingState / AggregatingState：让“状态本身就是聚合结果”

这两个状态适合“每来一条，就把它合并进当前结果”。例如求平均值，与其保存 100 万条订单，不如只保存 (sum, count)，最终 average = sum / count。

这是状态设计的一条重要原则：尽可能保存“足够回答问题的最小状态”，而不是保存全部历史原始数据。

5. Keyed State 与 Operator State 有什？

Keyed State 与 Operator State：状态“绑在谁身上”？

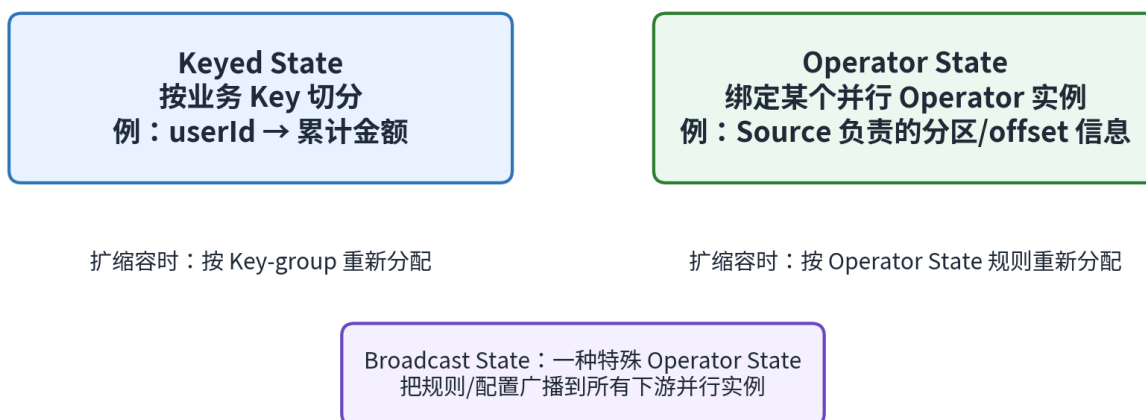


图 4 Keyed State 绑定业务 Key；Operator State 绑定并行算子实例

Keyed State 是大多数业务计算首先接触的状态；Operator State 更多用于 Source、Sink、Connector 或没有自然业务 Key 的场景。

维度	Keyed State	Operator State
绑定对象	业务 Key	某个 Operator 的并行实例
前置条件	通常需要 keyBy	不要求 KeyedStream
典型用途	用户累计、窗口、风控	Source 分区/offset、本地缓冲
扩缩容	按 Key-group 重分配	按 Operator State 规则重新分发
常见程度	业务逻辑中非常常见	较多见于连接器和特殊算子

Broadcast State 是一种特殊的 Operator State。典型场景是：一条小的“规则流”被广播给所有并行实例，例如实时风控规则、黑名单、动态配置。

6. State TTL : 如果永远不清理，状态会发生什么？

State TTL : 状态不能无限长大

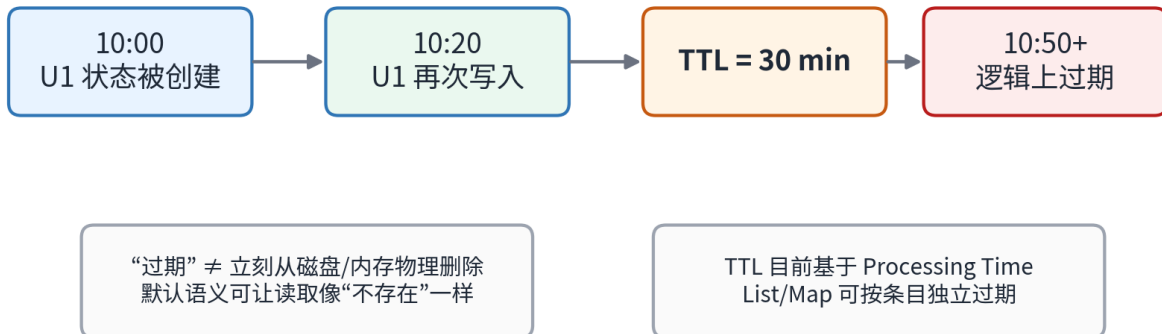


图 5 State TTL 控制状态生命周期，但“逻辑过期”与“物理清理”不是同一瞬间

一个运行几个月甚至几年的流任务，如果每见过一个用户就永久保留一份状态，状态规模可能只增不减。因此 Flink 支持给 Keyed State 配置 TTL (Time-To-Live)。

- TTL 可以应用于各种 Keyed State；List/Map 等集合状态可按条目独立过期。
- TTL 当前以 Processing Time 作为时间基准，不是 Event Time。
- 过期状态可以配置为读取时永不返回，即使它还没被后台物理删除。
- 过期数据的清理由读取、后台清理等机制尽力完成，因此“过期”不应理解成某一毫秒立即释放磁盘空间。

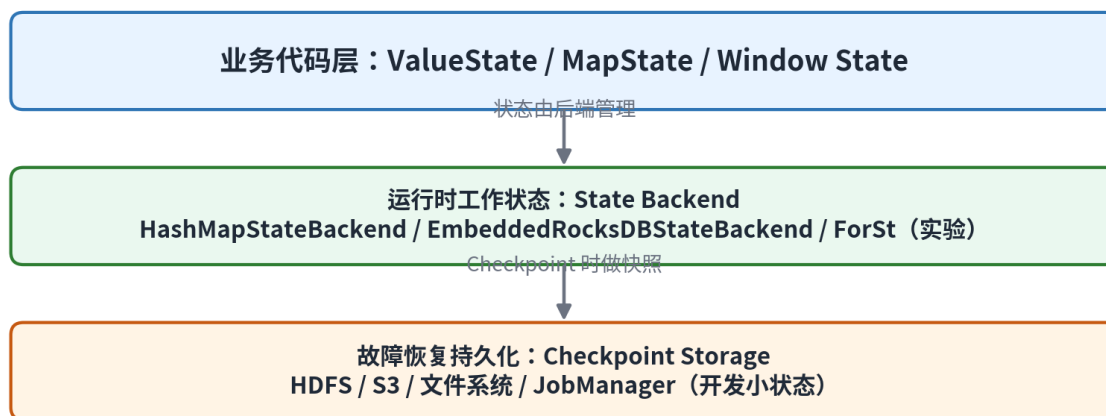
```
from pyflink.common import Time
from pyflink.datastream.state import StateTtlConfig

ttl = StateTtlConfig.new_builder(Time.hours(24)).build()
desc.enable_time_to_live(ttl)
```

TTL 解决的是“状态生命周期”问题，不是 Checkpoint 问题。TTL 决定某份状态多久以后视为无效；Checkpoint 决定故障后怎样恢复仍然有效的状态。

7. State 到底存在哪里？State Backend 是什？

不要混淆：State API、State Backend、Checkpoint Storage 是三层东西



核心区别：Backend 决定“运行时状态怎么放”；Checkpoint Storage 决定“快照最终存到哪里”。

图 6 State API、State Backend、Checkpoint Storage 是三层概念

初者最容易把三念混成一：“我代里的 ValueState”“TaskManager 运行时怎么存”“Checkpoint 最终写到哪里”。它们其实是三层。

层次	回答的问题	例子
State API	业务上要保存什么？	ValueState / MapState
State Backend	运行时工作状态怎么保存/访问？	HashMap / EmbeddedRocksDB / ForSt
Checkpoint Storage	快照持久化到哪里？	HDFS / S3 / filesystem

7.1 HashMapStateBackend：快，但受 JVM Heap 限制

HashMapStateBackend 的工作状态保存在 Java Heap 中。访问对象很快，但状态大会占用堆内存，并带来 GC 压力，因此更适合状态规模能够被内存承受的任务。

7.2 EmbeddedRocksDBStateBackend：状态可以大于内存

EmbeddedRocksDBStateBackend 使用本地 RocksDB 管理工作状态，可以借助磁盘承载更大的状态，并支持增量 Checkpoint。但代价是序列化、磁盘访问等开销更高。

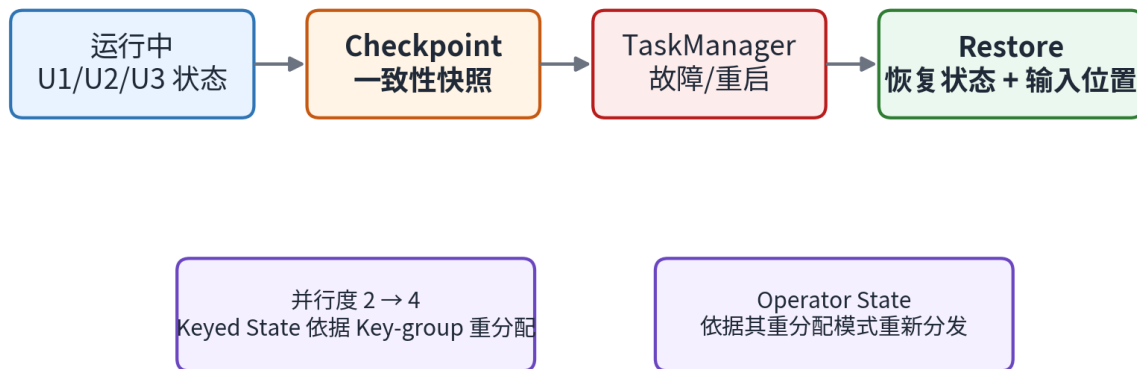
7.3 ForStStateBackend：面向远端/解耦状态的新方向

当前稳定文档还列出了 ForStStateBackend。它把底层 SST 文件放在远端文件系统，适合超大状态和云原生场景；但官方仍将其标为 experimental，因此本课程先理解概念，不把它作为入门实验主线。

不要用“内存后端 = Checkpoint 也只在内存”“RocksDB = Checkpoint 一定存在本地磁盘”这种方式理解。运行时 State Backend 与远端 Checkpoint Storage 已经是分离的概念。

8. State 与 Checkpoint：一个负责“记住”，一个负责“活下来”

State 如何支持故障恢复与扩缩容



“State”让流计算有记忆；“Checkpoint”让这份记忆在故障后可恢复。

图 7 State + Checkpoint 共同构成 Flink 有状态容错的基础

State 让算子拥有记忆，但如果 TaskManager 故障，只有内存里的记忆仍可能丢失。Checkpoint 会周期性地对分布式状态做一致性快照，并记录能够一致恢复的输入进度。

3. 任务运行：状态不断更新。
4. Checkpoint 到来：各算子把自己的状态纳入一致性快照。
5. 故障发生：Task/TaskManager 失败。
6. 作业重启：从最近成功 Checkpoint 恢复状态。
7. 输入 Source 从相匹配的位置继续消费，避免“状态是旧的但输入已经向前”的不一致。

所以：State 是“在线工作记忆”；Checkpoint 是“这份记忆的分布式恢复快照”。两者不是一回事。

9. 容，什能着走？

假设原来并行度为 2，U1/U2/U3/U4 的状态分布在两个 Subtask 上。现在并行度改为 4，如果状态只是普通进程内变量，你必须自己解决“哪些用户的状态应该搬到哪个新进程”。

Flink 的 Keyed State 按 Key-group 组织。恢复/Rescale 时，Key-group 可以重新分配给新的并行 Subtask，因此记录和对应状态仍然保持一致。

并行度 2：

```
Subtask0 -> key-groups 0..63
Subtask1 -> key-groups 64..127
```

并行度 4（示意）：

```
Subtask0 -> 0..31
Subtask1 -> 32..63
Subtask2 -> 64..95
Subtask3 -> 96..127
```

这也是为什么 Keyed State 比“自己在函数里建一个全局 HashMap”可靠得多：Flink 知道这份状态如何分区、快照、恢复和重新分配。

10. 一个完整案例：实时统计用户累计消费 + 超时清理

业务目标：订单流不断到来，按 user_id 实时输出累计消费额；如果某个用户 24 小时有活，不再期占用状态。

```
订单流
↓ key_by(user_id)
KeyedProcessFunction
↓ 读取 ValueState(total)
↓ total = old + amount
↓ update(total)
↓ 输出(user_id, total)
↓ TTL 24h 控制状态生命周期
```

如果 U1 依次产生 100、50、20 三笔订单，则逻辑变化如下：

事件	读取旧状态	写入新状态	输出
U1,100	None -> 0	100	U1,100
U1,50	100	150	U1,150
U2,80	None -> 0	80	U2,80
U1,20	150	170	U1,170

注意：U2 插入中间并不会影响 U1 的状态，因为 ValueState 会随当前 Key 自动切换。

11. 常见误区

- “State 就是一个普通变量。”——错。普通变量不具备按 Key 分片、Checkpoint、恢复和 Rescale 的系统语义。
- “keyBy 只是让相同 Key 去同一机器。”——不完整。它也建立了 Keyed State 的分区边界。

10. “ValueState 全局只有一个值。”——错。它是“每个 Key 一个逻辑值”。
11. “用了 State 就自动容错。”——不完整。要配合 Checkpoint 才能从故障中恢复一致状态。
12. “TTL 到点后一定瞬间释放磁盘。”——错。逻辑过期与物理清理是两个层次。
13. “RocksDB 就代表状态已经安全地持久化。”——错。运行时后端和远端 Checkpoint Storage 要区分。
14. “保存全部历史最保险。”——通常不是。状态越大，内存、磁盘、Checkpoint 和恢复成本越高。
15. “Operator State 比 Keyed State 更高级。”——不是等级关系，只是状态绑定对象和重分配语义不同。

12. 练习题

16. 实时统计每个设备最近一次温度，最适合哪种 State？为什么？
17. 每个用户需要保存“商品 ID -> 点击次数”，最适合哪种 State？
18. 为什么一个用户的 ValueState 在代码里不需要手工传 user_id？
19. State Backend 和 Checkpoint Storage 分别回答什么问题？
20. 如果任务运行一年，不设置 TTL 可能出现什么风险？
21. 为什么并行度从 2 改成 4 时，Keyed State 可以重新分配？

13. 参考答案

22. ValueState。每个设备只需要记住一个“最近温度”。
23. MapState。当前用户是外层 Key，商品 ID 是状态内部的小 Key。
24. 因为数据已经 keyBy，Flink Runtime 根据前理的 Key 自动切换状态分区。
25. State Backend 决定运行时工作状态如何保存/访问；Checkpoint Storage 决定快照持久化到哪里。
26. 状态不断膨胀，导致内存/磁盘占用增加、Checkpoint 变慢、恢复时间增长，最终可能拖垮长期作业。
27. Keyed State 按 Key-group 管理，恢复或扩缩容时可以把 Key-group 重新分配给新的并行 Subtask。

14. 本讲收束：把 State 压缩成 8 句话

28. State 是流处理系统的“记忆”。
29. Keyed State 是按业务 Key 分区的状态，通常与 keyBy 一起使用。
30. ValueState/ListState/MapState 的差别主要是“每个 Key 要保存什么结构”。
31. Operator State 绑定并行算子实例，而不是业务 Key。
32. TTL 管理状态生命周期，防止长期作业状态无限增长。
33. State Backend 决定运行时工作状态放在哪里、怎样访问。
34. Checkpoint Storage 决定故障恢复快照最终持久化到哪里。
35. State + Checkpoint + Key-group 重分配，构成 Flink 有状态计算、容错与扩缩容的核心基础。

下一讲：Checkpoint 与 Exactly-once。我们会回答——当几十个并行算子同时拥有 State，而数据又在不断流动时，Flink 怎样在“不停机”的情况下得到一张全局一致的状态快照？

15. 版本与资料说明

本讲按 Apache Flink 当前稳定文档中的经典 DataStream State 模型组织。当前稳定文档列出 HashMapStateBackend、EmbeddedRocksDBStateBackend，并包含仍处于实验阶段的 ForStStateBackend；DataStream API V2 也仍标记为实验阶段，因此入门课程继续以稳定、跨版本的状态概念为主。

官方参考：Apache Flink Stable Documentation - Working with State / State Backends / Fault Tolerance / PyFlink State API。